

Unit tests with pytest

Module 5 - Lesson 1

Overview

Tests verify that code behaves as expected and catch regressions. pytest is the standard Python framework: write plain functions that assert outcomes.

Key Concepts

- Test functions named `test_*` in files named `test_*.py` are discovered automatically.
- `assert` checks an expectation; pytest reports failures with rich diffs.
- Fixtures provide setup/teardown for tests that need data or resources.
- `Parametrize` runs the same test with multiple inputs.
- `Monkeypatch` or `mock` replaces external calls so tests run fast and offline.
- Aim for tests that are fast, isolated, and deterministic.

Example

```
def add(a, b):  
    return a + b  
  
# test_add.py  
import pytest  
from mymath import add  
  
@pytest.mark.parametrize("a,b,expected", [(1, 2, 3), (0, 0, 0), (-1, 1, 0)])  
def test_add(a, b, expected):  
    assert add(a, b) == expected
```

Summary

pytest turns assertions into a reliable safety net. Discovered automatically, extended by fixtures and `parametrize`, tests keep features working as you change code.

Practice Checklist

- Write tests for a small function using pytest.
- Add a fixture that supplies test data and a `parametrize` case.
- Run pytest and confirm failures point at the exact assertion.